IOT - 3

PROJEKAT 3

Veljko Veljović 1937

ELEKTRONSKI FAKULTET NIŠ

arduino

```
if (Serial.available() > 0) {
    char c = Serial.read();

    if (c == '\n') {
        threshold = inputString.toInt();
        Serial.print("Novi threshold: ");
        Serial.println(threshold);
        inputString = "";
    }
    else {
        inputString += c;
    }
}

delay(threshold);
    ei_printf("Recording...\n");
```

dodat je threshold za ocitavanje vremenskog intervala izmedju koga se prepoznaju glasovne komande

server.py

```python
from flask import Flask, jsonify, request
from flask_cors import CORS
from flask_socketio import SocketIO
from influxdb import InfluxDBClient
import paho.mqtt.client as mqtt
import serial


app = Flask(__name__)
CORS(app)
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')

influxClient = InfluxDBClient(
    host='localhost',
    port=8086,
    username='user',
    password='userpass',
    database='iotdb'
)

arduinoSerial = serial.Serial('COM5', 115200, timeout=1)
print("Arduino na portu: COM5")

def on_connect(client,userdata,flags,rc):
    print("Connected with code:", rc)
    client.subscribe("notification")
```

```python
def on_mqtt_message(client, userdata, msg):
    payload = msg.payload.decode()
    print("MQTT poruka:", payload)
    socketio.emit("new-data", {"data": payload})

mqttClient = mqtt.Client()
mqttClient.on_message = on_mqtt_message
mqttClient.on_connect = on_connect
mqttClient.connect("localhost", 1883, 60)
mqttClient.loop_start()


if __name__ == '__main__':
    socketio.run(app, host='0.0.0.0', port=5000)
```

```python
@app.route("/set-threshold", methods=["POST"])
def set_threshold():
    data = request.get_json()
    threshold = data.get("threshold")
    if arduinoSerial and threshold is not None:
        arduinoSerial.write(f"{threshold}\n".encode())
        print("Poslat threshold:", threshold)
        return jsonify({"status": "ok", "threshold": threshold})
    return jsonify({"status": "error", "message": "Arduino nije povezan ili threshold nije prosleđen"}), 400

@app.route("/set-anomaly", methods=['POST'])
def set_anomaly():
    data = request.get_json()
    anomaly = data.get("anomaly")
    mqttClient.publish("anomaly", anomaly)
    return jsonify({"status": "ok", "anomaly": anomaly})


@app.route('/simulate')
def simulate():
    socketio.emit("new-data", {"data": "X: 0.12 Y: -0.45 Z: 9.81 VERTICAL"})
    return "OK"

@app.route('/data')
def get_data():
    data = influxClient.query("SELECT * FROM imu_data")
    points = list(data.get_points())
    return jsonify(points)

@socketio.on('connect')
def handle_connect():
    print('Client connected')

ocketio.on('disconnect')
f handle_disconnect():
    print('Client disconnected')
```

react native - mobilna aplikacija

```
const App = () => {
  const [data, setData] = useState<Data[]>([]);
  const [toast, setToast] = useState<string | null>(null);
  const toastOpacity = useRef(new Animated.Value(0)).current;
  const [vremeOcitavanja, setVremeOcitavanja] = useState('');
  const [anomalija, setAnomalija] = useState('');

  const sendTime = () => {
    fetch(`${SERVER}/set-threshold`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ threshold: vremeOcitavanja }),
    }).catch(console.error);
  };

  const sendAnomaly = () => {
    fetch(`${SERVER}/set-anomaly`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ anomaly: anomalija }),
    }).catch(console.error);
  };
```

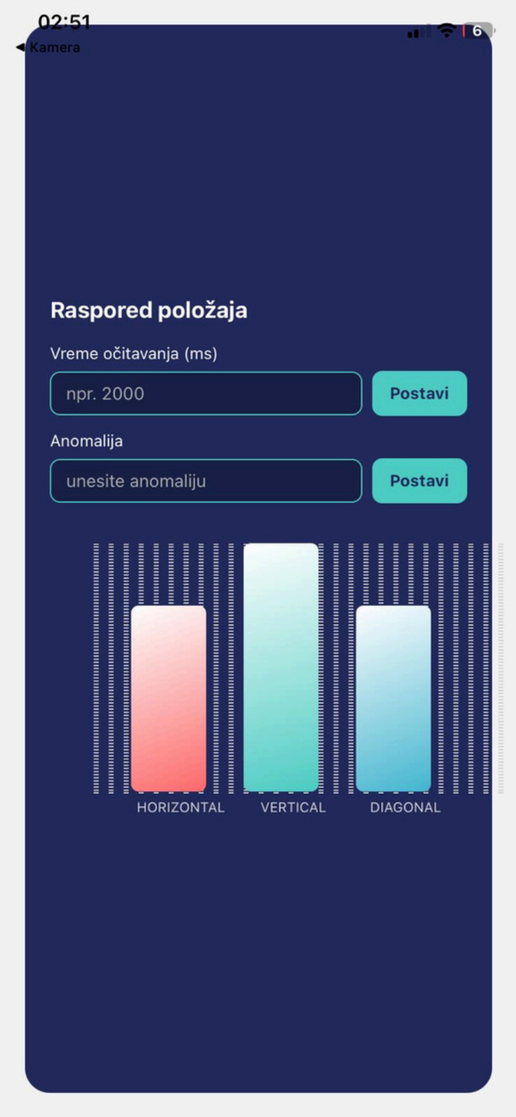
```
const showToast = (message: string) => {
  setToast(message);
  Animated.sequence([
    Animated.timing(toastOpacity, { toValue: 1, duration: 300, useNativeDriver: true }),
    Animated.delay(2500),
    Animated.timing(toastOpacity, { toValue: 0, duration: 400, useNativeDriver: true }),
  ]).start(() => setToast(null));
};

useEffect(() => {
  fetch(`${SERVER}/data`)
    .then(response => response.json())
    .then((json: Data[]) => setData(json))
    .catch(error => console.error(error));

  const socket = io(SERVER);

  socket.on('connect', () => console.log('Socket connected'));

  socket.on('new-data', (incoming: string) => {
    showToast(`Novo obaveštenje: ${incoming}`);
  });

  return () => { socket.disconnect(); };
}, []);
```

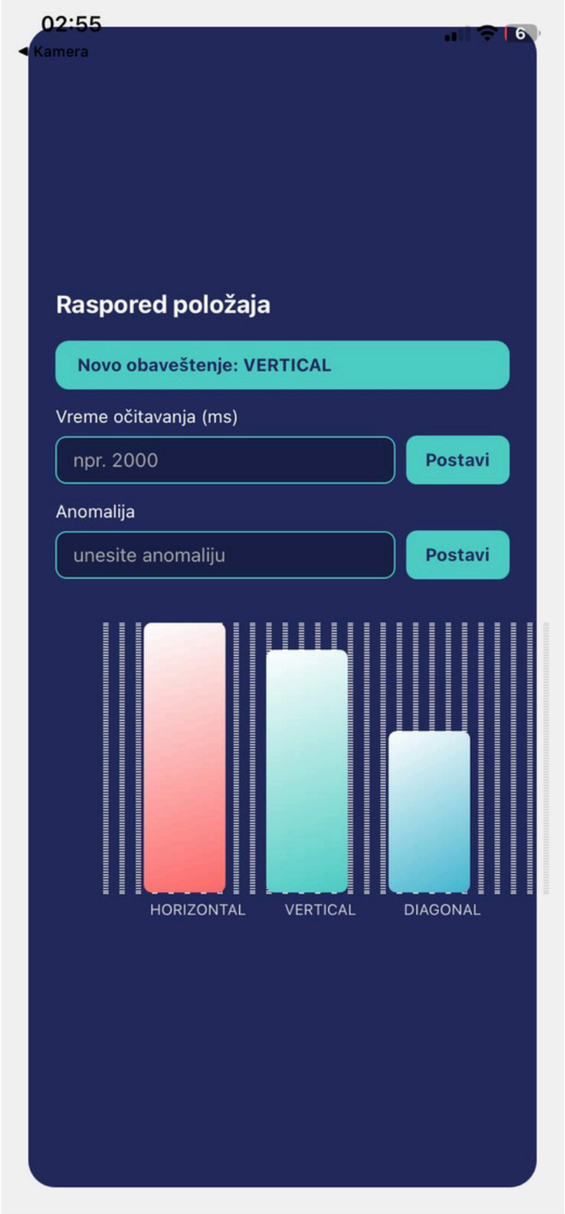
react native - mobilna aplikacija

```
const App = () => {
  const [data, setData] = useState<Data[]>([]);
  const [toast, setToast] = useState<string | null>(null);
  const toastOpacity = useRef(new Animated.Value(0)).current;
  const [vremeOcitavanja, setVremeOcitavanja] = useState('');
  const [anomalija, setAnomalija] = useState('');

  const sendTime = () => {
    fetch(`${SERVER}/set-threshold`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ threshold: vremeOcitavanja }),
    }).catch(console.error);
  };

  const sendAnomaly = () => {
    fetch(`${SERVER}/set-anomaly`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ anomaly: anomalija }),
    }).catch(console.error);
  };
```

```
const showToast = (message: string) => {
  setToast(message);
  Animated.sequence([
    Animated.timing(toastOpacity, { toValue: 1, duration: 300, useNativeDriver: true }),
    Animated.delay(2500),
    Animated.timing(toastOpacity, { toValue: 0, duration: 400, useNativeDriver: true }),
  ]).start(() => setToast(null));
};

useEffect(() => {
  fetch(`${SERVER}/data`)
    .then(response => response.json())
    .then((json: Data[]) => setData(json))
    .catch(error => console.error(error));

  const socket = io(SERVER);

  socket.on('connect', () => console.log('Socket connected'));

  socket.on('new-data', (incoming: string) => {
    showToast(`Novo obaveštenje: ${incoming}`);
  });

  return () => { socket.disconnect(); };
}, []);
```

react native - mobilna aplikacija





consumer.py

```python
def on_message(client, userdata, msg):
    message = msg.payload.decode()
    print("Primljeno:", message)
    if msg.topic == "anomaly":
        global anomaly
        anomaly = message
        print("Nova anomaly labela:", anomaly)
    elif msg.topic == "sensors/nano33":
        if "X:" in message:
            parts = message.split()
            x = float(parts[1])
            y = float(parts[3])
            z = float(parts[5])
            label = predict_orientation(x, y, z)
            print(x)
            print(y)
            print(z)
            print(label)
            if label == anomaly:
                print('alert')
                client.publish("notification", label)
            add_to_buffer(x,y,z,label)
```

Hvala na pažnji!